

Mobile Robotics Lab Manual

Lab Session 5: Introduction to Gazebo and RViz with ROS 2 using TurtleBot3

Azmeer Ali

University of Engineering and Technology (UET) Lahore

May 10, 2026

1 Objective

The objective of this lab session was to explore the simulation tools Gazebo and RViz within the ROS 2 Humble framework using the TurtleBot3 platform. The tasks involved visualizing, controlling, and navigating the robot in a simulated environment, utilizing Cartographer for Simultaneous Localization and Mapping (SLAM), and interacting with sensor data such as LiDAR and Odometry.

2 Methodology and Execution

2.1 Environment Setup

The environment was prepared by installing the required `turtlebot3` and `gazebo-ros-pkgs`. The TurtleBot3 model was exported as `burger`.

2.2 Simulation and Mapping

Gazebo was launched to simulate the physical robot in a 3D environment (`turtlebot3_world.launch.py`). Subsequently, Google Cartographer was launched alongside RViz to process LiDAR and IMU data to construct a 2D map in real-time. The robot was manually driven using a keyboard teleoperation node to explore the environment and generate a complete map, which was then saved utilizing the `nav2_map_server`.

2.3 Data Visualization and Recording

Within RViz, plugins for `LaserScan`, `Path`, `TF`, `Map`, and `Odometry` were configured. The `rosbag` utility was utilized to record all active topics during the teleoperation phase for offline analysis. The active nodes and their communication topology were visualized using `rqt_graph`.

3 ROS 2 Node Implementation

3.1 Task 8: Alternating Velocity Publisher

A custom ROS 2 node, `alternate_vel_node`, was developed to publish a `Twist` message to the `/cmd_vel` topic. A timer callback was utilized to alternate the linear x-velocity between 0.2m/s and 0.0m/s every 2 seconds.

```
1 import rclpy
2 from rclpy.node import Node
3 from geometry_msgs.msg import Twist
4
5 class AlternateVelocity(Node):
6     def __init__(self):
7         super().__init__('alternate_velocity_node')
8         self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)
9         self.timer = self.create_timer(2.0, self.timer_callback)
10        self.is_moving = False
11
12    def timer_callback(self):
13        msg = Twist()
14        if self.is_moving:
15            msg.linear.x = 0.0 # Stop
16            self.get_logger().info('Publishing: Stop (0.0 m/s)')
17        else:
18            msg.linear.x = 0.2 # Move forward
19            self.get_logger().info('Publishing: Forward (0.2 m/s)')
20
21        msg.angular.z = 0.0
22        self.publisher_.publish(msg)
23        self.is_moving = not self.is_moving
24
25    def main(args=None):
26        rclpy.init(args=args)
27        node = AlternateVelocity()
28        rclpy.spin(node)
29        node.destroy_node()
30        rclpy.shutdown()
31
32    if __name__ == '__main__':
33        main()
```

Listing 1: Alternating Velocity Publisher Node

3.2 Task 9: Odometry Subscriber

To track the robot's coordinates, an `odom_subscriber_node` was implemented. It subscribes to the `/odom` topic and extracts the X and Y coordinate data from the `nav_msgs/msg/Odometry` payload.

```
1 import rclpy
2 from rclpy.node import Node
3 from nav_msgs.msg import Odometry
4
5 class OdomSubscriber(Node):
6     def __init__(self):
7         super().__init__('odom_subscriber_node')
```

```

8         self.subscription = self.create_subscription(
9             Odometry, '/odom', self.listener_callback, 10)
10
11     def listener_callback(self, msg):
12         x_pos = msg.pose.pose.position.x
13         y_pos = msg.pose.pose.position.y
14         self.get_logger().info(f'Received Odom -> Position: X={x_pos:.3
15                                f}, Y={y_pos:.3f}')
16
17 def main(args=None):
18     rclpy.init(args=args)
19     node = OdometrySubscriber()
20     rclpy.spin(node)
21     node.destroy_node()
22     rclpy.shutdown()
23
24 if __name__ == '__main__':
25     main()

```

Listing 2: Odometry Subscriber Node

4 Deliverables and Output

[Note to student: Insert your screenshots in this section prior to final submission.]

- **RViz Visualization:** *(Insert screenshot showing TF, Odometry, LaserScan, and Path plugins)*
- **Node Communication:** *(Insert rqt_graph screenshot)*
- **Terminal Outputs:** *(Insert screenshots of the Task 8 and Task 9 terminal outputs running)*

5 Observations on Simulated Motion

While executing the teleoperation tasks, discrepancies between expected theoretical motion and simulated motion were observed. The physics engine in Gazebo accounts for factors like momentum, friction, and minor wheel slip. Because the `/odom` topic calculates position purely based on wheel encoder data (dead reckoning), these physical imperfections cause the odometry path in RViz to slowly drift away from the actual "ground truth" location of the robot in Gazebo over time. This highlights the necessity of SLAM algorithms like Cartographer, which use LiDAR to correct this odometry drift.

6 Conclusion

The lab provided comprehensive practical experience with essential robotic simulation toolchains. By bridging Gazebo's physics engine with RViz's visual debugging and Cartographer's SLAM capabilities, a complete mapping and navigation stack was explored. Furthermore, implementing custom publisher and subscriber nodes reinforced the core ROS 2 communication concepts, demonstrating how programmatic control interfaces with simulated hardware.